Round-Trip Refinement: A Training-Free Sampler for Next-Concept Language Models

Pierre Khoury
MSc Advanced Computing, King's College London
pierrecurrency@gmail.com

Abstract

Next-Concept Modeling (NCM) generates text coarse to fine: a multiscale residual VQ-VAE turns a sequence into twelve levels of discrete codes, and a transformer predicts one whole level per pass, so a 128-token sequence takes eleven generator passes. We find that the code stacks NCM samples are not the stacks its own tokenizer would assign to the text they decode to: re-encoding a generated sample rewrites 61–96% of the codes at each of the six coarsest levels, yet the re-encoded stack decodes back to the same text for 98% of samples. We propose *round-trip refinement*, a training-free sampler: decode the draft, re-encode the text with the tokenizer, keep the re-encoded coarse levels, and regenerate the fine levels with the unchanged generator. On the released TinyStories and LM1B checkpoints, using the authors' own sampling and evaluation code, one round lowers GPT-2 Large generative perplexity by 17% on TinyStories and 22% on LM1B (large) at equal unigram entropy, for six extra generator passes; two or three rounds reach −31% and −29%. Resampling the fine levels without re-encoding gives no gain, and re-quantizing the final latent instead of the text, a latent-space analogue of image scale-travel, recovers at most half of it. The gain is not a temperature effect in disguise: at matched corpus diversity refinement sits 10–15% below the temperature frontier at the released temperature, where an exact 11-pass twin without the round trip gains nothing, and it does not sharpen real human text. Combined with a per-level temperature schedule (hot coarse levels, cool fine levels), which on its own already moves the TinyStories frontier by up to 21%, refinement adds a further 4–10 points; the resulting samplers are about 21% more fluent than the released one at equal or higher diversity on held-out seeds of TinyStories and LM1B. All findings replicate on held-out seeds and across three LM1B model sizes, with gains that do not shrink with size; an LLM judge prefers refined stories to the released sampler's (0.63), and MAUVE against human text rises from 0.88 to 0.94–0.96. On Sudoku-hard, refinement solves 14–19 more of 1,000 puzzles while breaking at most one, but it does not help on easier puzzles, and with an exact verifier plain resampling is the better use of extra compute. On XSum, refined summaries are 16% more fluent with a small ROUGE gain; on WMT14, whose tokenizer round trip is lossy, refinement does not help. The image next-scale model VAR also samples non-canonical code stacks, but there the round trip through pixels does not improve FID. We also report what did not work on this budget: early exit, level skipping, level-dropout training, joint tokenizer–generator fine-tuning, training on the model's own errors, and distilling the refined sampler back into the generator, which matches the released perplexity only by giving up diversity. Code is available at https://github.com/PierreKhoury1/ncm-round-trip.

1 Introduction

Diffusion language models generate in parallel but need many denoising steps, each over the full sequence [28, 31]. Next-Concept Modeling [NCM; 13] takes a different route, borrowed from next-scale prediction in vision [12, 30]: a multiscale residual VQ-VAE encodes a 128-token sequence into K=12 levels of codes, from one code for the whole sequence to 128 codes at the finest level, and a transformer generates the levels coarse to fine, all codes of a level in parallel. NCM reaches the generative perplexity of strong diffusion baselines with an order of magnitude fewer function evaluations (NFE).

Each NCM level is sampled conditioned on the generator's own samples of the coarser levels. In training, by contrast, the coarser levels are the tokenizer's codes for real text, perturbed by uniform code replacement ["conditioning corruption"; 13, §3.4]. Residual quantization is many-to-one: many code stacks decode to the same tokens, and the tokenizer always produces one particular stack, the greedy residual quantization of its encoder output. A sampled stack need not be that one. We measure this directly: decoding a generated sample and re-encoding the text changes 61–96% of the codes at each of the six coarsest levels (20–72% at the finer ones), yet decoding the re-encoded stack gives back exactly the same text for 98% of samples (Section 5.4).

This suggests a simple repair. Once a draft exists, the tokenizer can tell us how *it* would have encoded that text. We keep the re-encoded coarse levels, which now form a stack of the kind the generator saw in training, and let the

generator redo the fine levels on top of them (Figure 1). The procedure needs no training, no new parameters and no change to the authors' sampler or decoder; it reuses the released generator, tokenizer and evaluation scripts as they are.

Contributions.

- A diagnosis: NCM's sampled code stacks are non-canonical (not the stacks the tokenizer assigns to their own decoded text), and re-encoding replaces them by canonical ones without changing the text (Section 5.4).
- Round-trip refinement, a training-free sampler with one knob, the restart level k (Section 3), and ablations showing that the re-encoding through text, not the extra passes, carries the gain.
- An evaluation on nine released checkpoints under the authors' protocols, with the temperature frontier, three diversity measures and exact 11-pass temperature twins as controls (Section 5): -17% to -31% gen-PPL on TinyStories and -22% to -29% on LM1B large, 10–15% beyond the temperature frontier, and clear failure cases (translation with a lossy tokenizer, easy Sudoku).
- Evidence beyond Gen. PPL: held-out seeds, an LLM judge, MAUVE against human text, a trend over three model sizes, and a reasoning task (Sudoku) where refinement fixes puzzles without breaking solved ones (Sections 5.6–5.8).
- Negative results for six training-side ideas, including the two the NCM paper lists as future work, so that others need not repeat them (Section 6).

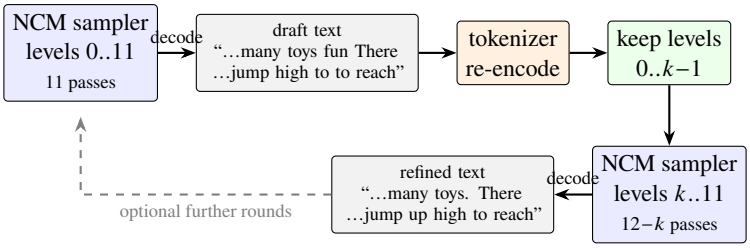


Figure 1: Round-trip refinement (RT_k). The draft is decoded to text and re-encoded by the tokenizer; the re-encoded coarse levels replace the sampled ones and the generator regenerates levels $k..11$. The text is a real example (TinyStories, seed 1, sample 4; full text in Appendix B).

2 Background: Next-Concept Modeling

Tokenizer. An encoder E maps a token sequence $x \in \mathcal{V}^L$ (L=128) to features $z \in \mathbb{R}^{L\times d}$. A multiscale residual quantizer with level lengths $\ell_0, \ldots, \ell_{K-1}$ (for TinyStories $1, 8, 16, 24, 32, 40, 52, 64, 76, 92, 108, 128$) then picks codes greedily,

$$r_k = \arg\min_j \left\| \mathrm{Down}_{\ell_k}(z - \hat{z}_{k-1}) - e_{k,j} \right\|, \qquad \hat{z}_k = \hat{z}_{k-1} + \phi_k\big(\mathrm{Up}_L(e_{k,r_k})\big), \tag{1}$$

with codebooks e_k of 4,096 entries and $\hat{z}_{-1}$=0. A decoder D maps $\hat{z}_{K-1}$ back to token logits. We write $Q(E(x))$ for the full code stack of x and $\mathrm{dec}(r)$ for decoding a stack to tokens with the authors' token picker (argmax, with frequency and adjacency penalties on LM1B).

Generator. The NCM transformer predicts all ℓ_k codes of level k in one pass from the reconstruction $\hat{z}_{k-1}$ resized to ℓ_k positions, with block-diagonal attention so that pass k processes only ℓ_k positions. Unconditional generation draws r_0 from a unigram prior and then runs $K-1$=11 passes. Training uses teacher forcing on $Q(E(x))$ for real text, with a share of the input codes replaced uniformly at random (mean rate 0.5, decreasing with level), while targets stay clean.

Algorithm 1 Round-trip refinement with restart levels $k_1, \ldots, k_m$

$r \leftarrow$ NCM-SAMPLE() ▷ draft: r_0 from the prior, then $K-1$ passes
$x \leftarrow \mathrm{dec}(r)$
for $k \in (k_1, \ldots, k_m)$ **do**
  $r' \leftarrow Q(E(x))$ ▷ re-encode the current text
  $r \leftarrow$ NCM-SAMPLE(levels $0..k-1$ fixed to $r'_{0..k-1}$) ▷ $K-k$ passes
  $x \leftarrow \mathrm{dec}(r)$
end for
return x

3 Round-Trip Refinement

Algorithm 1 gives the sampler. One round with restart level k, written RT_k, costs $K-k$ extra generator passes plus one tokenizer encoder pass. Multiple rounds are written $\mathrm{RT}_5{\times}2$ (two rounds from level 5) or $\mathrm{RT}_3{+}\mathrm{RT}_6{+}\mathrm{RT}_9$ (three rounds from successively finer levels). Because pass k processes ℓ_k positions, regenerating levels 6..11 reprocesses 520 of the 640 positions of a full sample; FLOPs therefore grow by about 1.8× for RT_6 and 3.3× for three rounds, somewhat more than the NFE ratios of 1.5× and 2.6× suggest.

Implementation. We do not reimplement sampling. The authors' `generate` calls a per-level sampling function once per level; we wrap that function so that, for levels below k, it returns the re-encoded codes instead of sampling (Appendix A). Code embeddings, level encodings, heads, attention masks and the token picker are therefore exactly the released ones. As a check, forcing *every* level to the re-encoded stack reproduces the draft text exactly on TinyStories (8/8 samples tested), 27/32 on XSum and only 15/32 on WMT14, whose tokenizer round trip is lossy (Section 5.3).

Ablations. Two controls isolate where the gain comes from. *Resample*$_k$ keeps the draft's own codes for levels below k and regenerates the rest; it costs the same as RT_k but involves no re-encoding, so it draws the fine levels from the same conditional distribution as the draft did. *LatentRQ*$_k$ re-quantizes the draft's latent $\hat{z}_{K-1}$ with Eq. (1) instead of decoding and re-encoding the text; it is the text analogue of scale-travel in DiverseVAR [21].

4 Experimental setup

Checkpoints and sampling. We use the released NCM checkpoints for TinyStories [7] (88.6M generator), LM1B [5] small and large, XSum [20] and WMT14 de–en [2], with the released evaluation configurations: temperature 1, no top-k/top-p and no guidance for TinyStories and LM1B (LM1B with the configured frequency and adjacency penalties in the token picker), greedy decoding for XSum and WMT14. Refinement rounds use the same settings as the draft unless stated otherwise.

Metrics. For TinyStories we report generative perplexity (Gen. PPL) under GPT-2 Large [27] and the mean per-sample unigram entropy H of GPT-2 tokens, as in the NCM paper; for LM1B we use the authors' FLM protocol (corpus perplexity over the first 128 GPT-2 tokens of the decoded ids, entropy over raw ids). Because Gen. PPL can be lowered by reducing diversity [8, 24], we add corpus distinct-3 (unique word trigrams over all trigrams), cross-sample 4-gram overlap (xs-4, share of a sample's 4-grams that occur in another sample; lower is more diverse), and the rate of adjacent doubled words, and we compare every sampler with the *temperature frontier*: the released sampler at $\tau \in \{1, 0.95, 0.9, 0.85, 0.8, 0.7\}$, interpolated at the same distinct-3 (TinyStories) or entropy (LM1B). XSum is scored with ROUGE-1/2/L [17] plus GPT-2 Gen. PPL of the summaries, WMT14 with sacreBLEU [23] and chrF. Unconditional results average three seeds of 256 samples each. All samplers of a seed share their level-0 draws; drafts are identical across samplers only for the first batch of each seed (samples 0–127 on TinyStories, 0–31 on LM1B), because refinement rounds consume random numbers before the second batch. Conditional results use the first 1,000 test documents.

Reproduction. Our pipeline gives 16.15 ± 0.17 Gen. PPL for the released TinyStories sampler (seeds 1–3) and 16.32 on seeds 4–6; the closest number in the NCM paper is 15.41, from its block-diagonal attention ablation, which is not the released evaluation run. All comparisons below are against our own reproduction, under identical code and seeds.

5 Results

5.1 TinyStories

Sampler	NFE	Gen. PPL ↓	Δ	H	dist-3 ↑	xs-4 ↓	dup% ↓	vs. τ frontier
released (τ=1)	11	16.15±0.17	–	3.980	0.749	0.162	0.39	–
τ=0.95	11	15.13±0.27	-6.3%	3.980	0.741	0.173	0.36	–
τ=0.9	11	14.64±0.15	-9.4%	3.979	0.735	0.179	0.36	–
τ=0.85	11	13.75±0.41	-14.9%	3.968	0.723	0.188	0.32	–
τ=0.8	11	13.34±0.25	-17.4%	3.968	0.719	0.191	0.35	–
τ=0.7	11	12.59±0.18	-22.1%	3.962	0.699	0.213	0.36	–
RT_9	14	15.13±0.15	-6.4%	3.980	0.747	0.165	0.33	-4.0%
RT_8	15	14.76±0.31	-8.6%	3.980	0.744	0.168	0.28	-6.0%
RT_7	16	13.84±0.33	-14.3%	3.974	0.740	0.171	0.24	-8.3%
RT_6	17	13.36±0.12	-17.3%	3.974	0.740	0.170	0.23	-12.4%
RT_5	18	12.90±0.47	-20.1%	3.975	0.731	0.181	0.31	-10.9%
RT_4	19	12.90±0.31	-20.2%	3.976	0.727	0.184	0.33	-8.8%
RT_3	20	13.24±0.15	-18.0%	3.969	0.720	0.195	0.39	-2.2%
RT_6×2	23	12.64±0.34	-21.8%	3.973	0.730	0.178	0.19	-12.3%
RT_4+RT_8	23	12.34±0.18	-23.6%	3.969	0.724	0.187	0.24	-10.1%
RT_5×2	25	11.70±0.31	-27.6%	3.970	0.718	0.191	0.25	-12.9%
RT_3+RT_6+RT_9	29	11.17±0.24	-30.9%	3.963	0.708	0.205	0.22	-13.7%
RT_6, $\tau_{\geq 6}$=0.8	17	12.96±0.17	-19.8%	3.969	0.739	0.171	0.23	-14.4%
RT_5, $\tau_{\geq 6}$=0.8	18	12.48±0.44	-22.7%	3.974	0.729	0.182	0.27	-13.1%
RT_5×2, $\tau_{\geq 6}$=0.9	25	11.56±0.16	-28.4%	3.969	0.717	0.193	0.27	-13.1%
human text (validation)	–	8.51	–	–	0.775	0.139	<0.01	–

Table 1: TinyStories, released checkpoint, GPT-2 Large, 3 seeds × 256 samples (mean ± s.d. over seeds). Δ: change in Gen. PPL against the released sampler. Last column: Gen. PPL relative to the temperature curve (τ=0.7..1.2) interpolated at the same corpus distinct-3, per seed then averaged (negative = better than changing the temperature). Every refinement variant beats the released sampler on all three seeds.

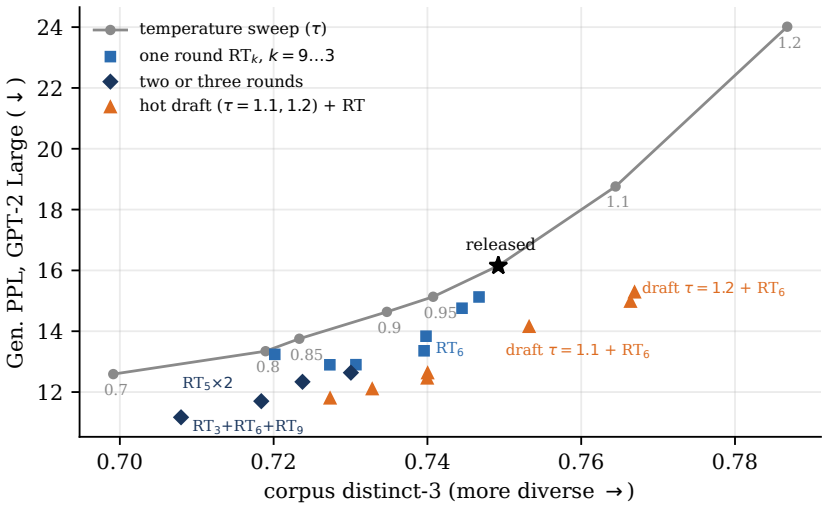


Figure 2: TinyStories: Gen. PPL against corpus distinct-3. Lowering the temperature buys fluency with diversity along the grey curve; round-trip refinement lands below it at every restart level, and further rounds move along a lower curve.

Table 1 and Figure 2 give the main result. One round from level 6 lowers Gen. PPL from 16.15 to 13.36 (−17.3%, all seeds) with unigram entropy essentially unchanged (3.974 vs. 3.980). Restarting from coarser levels helps up to k=4–5 (−20%); restarting from level 3 rewrites too much of the content and the gain shrinks. More

rounds keep improving: $RT_5 \times 2$ reaches 11.70 (−27.6%) and RT_3+RT_6+RT_9 reaches 11.17 (−30.9%), closing 65% of the gap to human text (8.51).

Not a temperature effect. Lower temperature also lowers Gen. PPL: τ=0.8 gives 13.34, the same as RT_6. But it does so by giving up diversity, to distinct-3 0.719 and cross-sample overlap 0.191, where RT_6 keeps 0.740 and 0.170. Interpolating the temperature curve (τ from 0.7 to 1.2) at each sampler's own distinct-3, single rounds are 6–12% better than temperature for k=5..8 and multiple rounds 10–14% better; sampling levels 6–11 at τ=0.8 ($\tau_{\geq 6}$=0.8, in the draft and in the round) gives the largest single-round margin, −14.4%. Refinement also removes errors that temperature does not touch: adjacent doubled words ("to to", "quickly quickly"), a typical failure of predicting a whole level in parallel, fall from 0.39% of words to 0.23% under RT_6, while every temperature setting leaves them at 0.32–0.36%.

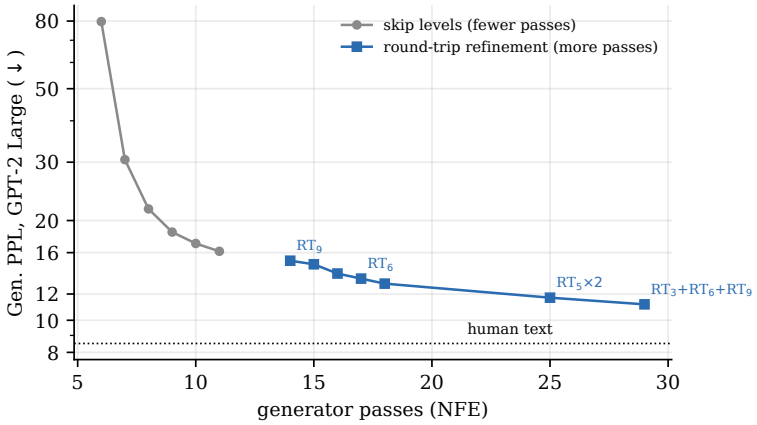


Figure 3: Quality against generator passes on TinyStories (GPT-2 Large). Grey: the released generator with middle levels skipped (fewer passes), which degrades quickly. Blue: round-trip refinement, which turns extra passes into steady gains. Dotted: human-written validation openings.

Compute. Figure 3 places refinement on a quality–compute curve. Skipping levels of the released sampler to save passes costs +6% for one pass and +89% for four; refinement spends 3–18 extra passes for −6% to −31%. NCM's speed advantage over diffusion LMs is 10–31× in wall-clock [13], so even the three-round sampler (29 passes, about 3.3× the FLOPs) stays far cheaper than the 128–1,024-step diffusion samplers NCM is compared with.

5.2 LM1B

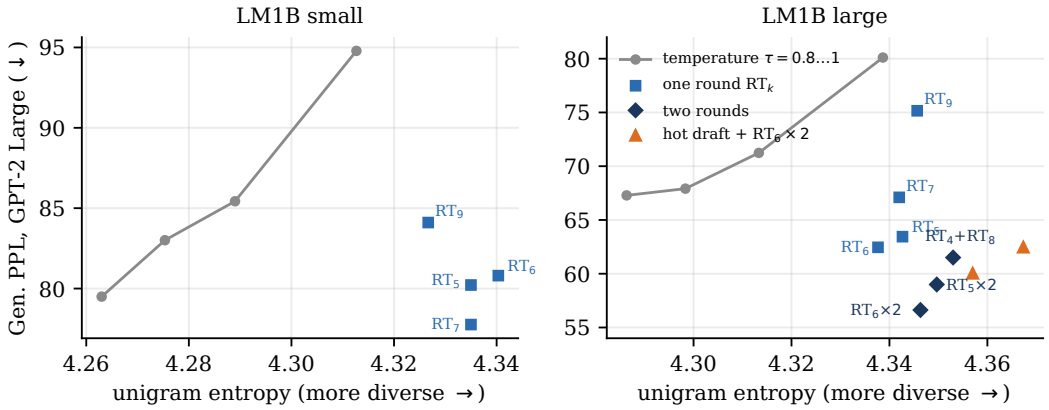


Figure 4: LM1B: Gen. PPL against unigram entropy. Temperature (grey) trades entropy for perplexity; refinement moves straight down, and on the large model two rounds reach −29% at slightly *higher* entropy.

Sampler	NFE	Gen. PPL ↓	Δ	H	dist-3 ↑	xs-4 ↓	seeds better
LM1B small							
released (τ=1)	11	94.79±2.56	–	4.313	0.864	0.055	–
τ=0.9	11	85.43±0.55	-9.9%	4.289	0.832	0.075	3/3
τ=0.85	11	83.01±1.84	-12.4%	4.275	0.814	0.084	3/3
τ=0.8	11	79.50±1.32	-16.1%	4.263	0.799	0.096	3/3
RT_9	14	84.11±2.26	-11.3%	4.327	0.855	0.058	3/3
RT_7	16	77.76±1.72	-18.0%	4.335	0.837	0.070	3/3
RT_6	17	80.80±0.39	-14.8%	4.340	0.838	0.073	3/3
RT_5	18	80.22±3.28	-15.4%	4.335	0.833	0.076	3/3
LM1B medium							
released (τ=1)	11	87.47±2.16	–	4.339	0.895	0.043	–
τ=0.9	11	75.46±1.56	-13.7%	4.309	0.870	0.057	3/3
τ=0.85	11	72.40±2.55	-17.2%	4.298	0.859	0.065	3/3
τ=0.8	11	70.22±1.40	-19.7%	4.285	0.848	0.071	3/3
RT_7	16	70.30±1.25	-19.6%	4.341	0.880	0.049	3/3
RT_6	17	69.43±0.78	-20.6%	4.342	0.873	0.056	3/3
RT_6×2	23	62.30±0.25	-28.8%	4.342	0.860	0.062	3/3
LM1B large							
released (τ=1)	11	80.11±0.34	–	4.339	0.895	0.043	–
τ=0.9	11	71.24±0.71	-11.1%	4.313	0.871	0.057	3/3
τ=0.85	11	67.91±1.76	-15.2%	4.298	0.861	0.064	3/3
τ=0.8	11	67.29±0.94	-16.0%	4.286	0.855	0.068	3/3
RT_9	14	75.16±1.68	-6.2%	4.346	0.892	0.045	3/3
RT_7	16	67.10±1.15	-16.2%	4.342	0.885	0.047	3/3
RT_6	17	62.46±1.63	-22.0%	4.338	0.878	0.052	3/3
RT_5	18	63.45±1.15	-20.8%	4.343	0.869	0.060	3/3
RT_6×2	23	56.63±0.88	-29.3%	4.346	0.869	0.059	3/3
RT_5×2	25	59.00±0.27	-26.3%	4.350	0.855	0.068	3/3
RT_4+RT_8	23	61.51±1.26	-23.2%	4.353	0.859	0.067	3/3
RT_6, $\tau_{\geq 6}$=0.9	17	58.97±0.48	-26.4%	4.325	0.868	0.058	3/3

Table 2: LM1B, released small and large checkpoints, authors' FLM protocol (GPT-2 Large), 3 seeds × 256 samples. Diversity measured on BERT word-piece ids. Refinement improves Gen. PPL at equal or higher entropy; temperature cannot.

The result carries over to LM1B, a different domain (news sentences), tokenizer (BERT word pieces) and evaluation protocol (Table 2, Figure 4). On the small model one round lowers Gen. PPL by 11–18%, on the large model by 6–22%, on every seed and at equal or higher entropy, whereas τ=0.8 reaches −16% only by dropping entropy from 4.339 to 4.286. Two rounds from level 6 reach 56.63 on the large model (−29.3%) at entropy 4.346. The released medium model behaves the same way (RT_6 −20.6%, RT_6×2 −28.8% at entropy 4.342 vs. 4.339), and the gain of RT_6 does not shrink with model size: −14.8% (small), −20.6% (medium), −22.0% (large), which suggests the effect is not an artefact of a weak generator. Entropy is the least sensitive diversity axis on LM1B; measured on distinct-3 of the raw ids, RT_6 is 13–17% and RT_6×2 17–21% below the temperature curve on every seed (1–6), and the hot-draft sampler of Section 5.5 reaches −21% at the released sampler's distinct-3 (0.894 vs. 0.896).

5.3 Conditional generation

On XSum (Table 3), one round from level 3 raises ROUGE-1/2/L by +0.25/+0.25/+0.33 and lowers the Gen. PPL of the summaries from 82.4 to 69.5 (−16%); two rounds overshoot. On WMT14 refinement lowers BLEU at every restart level, least for the coarsest restart (RT_4: 20.45 vs. 20.51) and by over 4 points for RT_6. The sanity check of Section 3 explains why: re-encoding a WMT14 output and decoding it again, with no regeneration at all, already changes 17 of 32 outputs. When the tokenizer cannot reproduce its own outputs, the round trip injects errors, and in translation exact wording is what the metric rewards. Round-trip refinement therefore needs a tokenizer whose round trip is (near-)lossless on generated text; on TinyStories it is exact.

Task	Sampler	NFE	R-1	R-2	R-L	Gen. PPL
XSum	released (τ=1)	8	34.63	12.84	27.77	82.4
XSum	RT_7	10	34.78	12.90	27.84	75.7
XSum	RT_6	11	34.79	12.87	27.89	71.4
XSum	RT_5	12	34.84	12.89	27.90	69.9
XSum	RT_4	13	34.85	12.98	27.93	69.1
XSum	RT_3	14	34.88	13.09	28.10	69.5
XSum	$RT_5\times2$	16	34.57	12.69	27.59	82.5
WMT14	released (τ=1)	10	BLEU 20.51, chrF 48.04			len. 0.90
WMT14	RT_8	13	BLEU 19.05, chrF 47.09			len. 0.98
WMT14	RT_7	14	BLEU 19.01, chrF 47.35			len. 0.94
WMT14	RT_6	15	BLEU 16.38, chrF 46.39			len. 1.07
WMT14	RT_5	16	BLEU 17.21, chrF 46.80			len. 1.01
WMT14	RT_4	17	BLEU 20.45, chrF 47.57			len. 0.95
WMT14	$RT_6\times2$	20	BLEU 17.44, chrF 46.52			len. 1.05

Table 3: Conditional tasks, released checkpoints, greedy decoding as configured, 1,000 test documents. XSum: refinement makes summaries more fluent with a small ROUGE gain; two rounds hurt. WMT14 de–en: refinement lowers BLEU, because the tokenizer round trip itself changes the output.

5.4 Why the round trip helps

Sampled stacks are non-canonical. Figure 5 shows, for 768 TinyStories drafts, the share of codes per level that change when the draft's stack is replaced by the tokenizer's stack for its decoded text: 96% of the single level-0 codes, 61–84% at levels 1–5, and 20–72% at the finer levels. Yet decoding the re-encoded stack reproduces 751 of the 768 drafts exactly (97.8%, and 99.98% of tokens). The generator's stack and the tokenizer's stack are two different descriptions of the same text, and only the second is the kind of stack the generator conditions on in training, where its inputs are tokenizer stacks with uniform noise.

The re-encoding is what helps. Table 8 separates the ingredients. Regenerating levels 6–11 conditioned on the draft's own coarse codes (Resample_6) costs the same six passes as RT_6 but gives no gain at all (16.43 vs. 16.15 on TinyStories, +0.6% on LM1B): it only draws the fine levels again from the conditional distribution that produced them. Re-quantizing the draft's latent with Eq. (1) instead of re-encoding its text (LatentRQ_6, a final-latent analogue of image scale-travel, which re-quantizes a partial map at an intermediate scale) changes the coarse codes almost as much, except level 0, which it keeps 72% of the time, and recovers part of the gain: −11.5% on TinyStories and −17.9% on LM1B, but with lower diversity, so that at matched diversity it is −6.8% and −7.7% against −12.4% and −20.9% for RT_6. Going through the decoded text, which discards whatever in the latent does not affect the output and lets the encoder describe the text afresh, roughly doubles the matched-diversity gain. The same ordering holds with two rounds ($\text{Resample}_5\times2$ −0.2%, $\text{LatentRQ}_5\times2$ −19.9%, $RT_5\times2$ −27.6%).

5.5 Spending the gain on diversity

Because refinement lowers Gen. PPL at almost constant diversity, it can be paired with a *hotter* draft to land on a better frontier (Table 8, Figure 2). With the draft at τ=1.1 and one round from level 6 at τ=1, Gen. PPL is 14.15 (−12.4%, all seeds) while distinct-3 rises from 0.749 to 0.753, cross-sample overlap falls from 0.162 to 0.156, unigram entropy is unchanged (3.981) and doubled words fall from 0.39% to 0.26%: the sampler is better than the released one on every axis we measure, at the cost of six passes. With the draft at τ=1.2 and the regenerated levels at τ=0.8, samples are clearly more diverse than the released sampler's (distinct-3 0.766, overlap 0.143) and still 7.3% more fluent; the temperature curve at that diversity is at 19.2, a frontier gain of 22%. Hot drafts with two or three rounds reach −22% to −27% Gen. PPL at 17–18% below the frontier. On LM1B large, a τ=1.2 draft with $RT_6\times2$ gives 62.47 (−22.0%) at higher entropy (4.367 vs. 4.339), the same distinct-3 (0.894 vs. 0.895) and lower cross-sample overlap (0.040 vs. 0.043). The hot-draft settings come from a small grid (draft $\tau \in \{1.1, 1.2\}$, three round schedules) chosen on seeds 1–3 and confirmed on held-out seeds below. Part of these gains comes from the per-level temperature schedule itself rather than from the round trip; Section 5.9 separates the two with exact 11-pass twins.

5.6 Held-out seeds

All TinyStories settings above were chosen on seeds 1–3. Table 4 re-runs the main samplers, the ablations and the full temperature curve on three fresh seeds (4–6, 3×256 samples). Every conclusion replicates, and the matched-diversity gains are, if anything, slightly larger: -14.5% for RT_6, -15.3% for $RT_5{\times}2$, and -21.4% for a τ=1.2 draft refined twice with the regenerated levels at τ=0.8, which lowers Gen. PPL by 23.6% at essentially the released sampler's diversity (distinct-3 0.742 vs. 0.748, cross-sample overlap 0.165 vs. 0.165). Resampling without re-encoding again does nothing (-1.2%). On LM1B large, held-out seeds give -20.2% (RT_6) and -27.1% ($RT_6{\times}2$), against -22.0% and -29.3% on seeds 1–3.

Sampler	NFE	Gen. PPL ↓	Δ	dist-3 ↑	xs-4 ↓	vs. τ frontier
released (τ=1)	11	16.32±0.19	–	0.748	0.165	–
τ=0.8	11	13.44±0.22	-17.6%	0.717	0.195	–
RT_6	17	13.23±0.08	-18.9%	0.737	0.173	-14.5%
$RT_5{\times}2$	25	11.94±0.11	-26.8%	0.725	0.185	-15.3%
RT_3+RT_6+RT_9	29	11.50±0.10	-29.5%	0.715	0.199	-14.3%
RT_6, $\tau_{\geq 6}$=0.8	17	12.98±0.10	-20.5%	0.735	0.174	-14.8%
RT_6, draft τ=1.1	17	13.96±0.04	-14.5%	0.746	0.166	-13.5%
RT_6, draft τ=1.2, $\tau_{\geq 6}$=0.8	17	14.46±0.08	-11.4%	0.755	0.153	-17.7%
$RT_5{\times}2$, draft τ=1.2, $\tau_{\geq 6}$=0.8	25	12.47±0.57	-23.6%	0.742	0.165	-21.4%
$Resample_6$	17	16.12±0.35	-1.2%	0.747	0.164	-0.7%
$LatentRQ_6$	17	14.27±0.08	-12.5%	0.739	0.172	-8.5%

Table 4: TinyStories on held-out seeds 4–6 (GPT-2 Large, 3×256). The frontier is the temperature curve (τ=0.7..1.2) measured on the same seeds.

5.7 Beyond generative perplexity: an LLM judge and MAUVE

Gen. PPL rewards text that GPT-2 finds predictable. Two further measures check that refined stories are actually better and closer to human writing. First, an open-weights LLM judge (Qwen2.5-7B-Instruct; 26) compares paired stories (sample i of two samplers starts from the same level-0 draw) in both orders and gives the probability that each is more fluent, grammatical and coherent; averaging the two orders removes its strong preference for the story shown first (0.78–0.91). Second, MAUVE [22] measures the gap between each sampler's 768 stories and 768 human-written validation openings in GPT-2 Large feature space.

X	Y	P(Y preferred)	95% CI	both orders Y / X (%)	note
released	RT_6	0.634	[0.617, 0.651]	30.6 / 3.3	main comparison
released	RT_3+RT_6+RT_9	0.587	[0.573, 0.601]	19.4 / 2.1	
released	RT_6, draft τ=1.1	0.531	[0.518, 0.544]	11.1 / 5.2	more diverse than released
τ=0.8	$RT_5 \times 2$	0.529	[0.515, 0.542]	11.8 / 6.0	equal distinct-3
τ=0.8	RT_6	0.506	[0.493, 0.519]	8.7 / 7.8	equal Gen. PPL
$LatentRQ_6$	RT_6	0.557	[0.539, 0.574]	20.7 / 9.8	text vs. latent round trip
released	τ=0.8	0.537	[0.523, 0.551]	12.6 / 6.0	temperature alone

Table 5: Blind pairwise preferences of Qwen2.5-7B-Instruct over 768 paired TinyStories samples (seeds 1–3), averaged over both presentation orders. "Both orders": share of pairs where the judge prefers the same story in both orders.

The judge prefers RT_6 over the released sampler (0.634; it picks RT_6 in both orders for 30.6% of pairs and the released story for 3.3%), three rounds likewise (0.587), and the hot-draft sampler that is more diverse than the released one still wins (0.531). At equal distinct-3, two rounds beat the temperature-lowered sampler (0.529, interval above 0.5), and the text round trip beats latent re-quantization (0.557). At equal Gen. PPL, however, RT_6 and τ=0.8 tie (0.506): for the judge, the advantage of refinement over temperature at that point is the diversity it keeps, not better individual stories. MAUVE (Table 6) rises from 0.882 for the released sampler to 0.94–0.955 for refinement, the best value for $RT_5{\times}2$; lowering the temperature also raises it (0.943–0.946), so MAUVE confirms that refined samples are closer to human text but does not by itself separate refinement from temperature.

Sampler	MAUVE ↑	Gen. PPL	dist-3
released (τ=1)	0.882±0.012	16.15	0.749
τ=0.9	0.946±0.002	14.64	0.735
τ=0.8	0.943±0.011	13.34	0.719
Resample_6	0.913±0.004	16.43	0.750
LatentRQ_6	0.925±0.005	14.29	0.741
RT_6	0.941±0.004	13.36	0.740
RT_6, $\tau_{\geq 6}$=0.8	0.946±0.013	12.96	0.739
$\text{RT}_5\times 2$	0.955±0.005	11.70	0.718
RT_3+RT_6+RT_9	0.945±0.006	11.17	0.708
RT_6, draft τ=1.1	0.929±0.005	14.15	0.753
RT_6, draft τ=1.2, $\tau_{\geq 6}$=0.8	0.952±0.001	14.98	0.766

Table 6: MAUVE against 768 human TinyStories openings (GPT-2 Large features, mean ± s.d. over three k-means seeds; differences between sample sets of about 0.03 are within sampling noise).

5.8 Reasoning: Sudoku

NCM's Sudoku results [13] make it a test of whether refinement only polishes surface text or also helps a task with a single correct answer. On the released Sudoku-hard model (1,000 test puzzles, greedy decoding as configured), every restart level solves more puzzles than the released sampler (Table 7): from 84.1% to 85.5% with RT_6 (14 newly solved puzzles, none broken; exact McNemar p=1.2 × 10^{-4}) and 85.9% with three rounds (19 solved, 1 broken). Here the round trip changes only 2–20% of the codes per level, far less than on stories, and greedy rounds converge quickly ($\text{RT}_6\times 2$ and $\text{RT}_6\times 4$ are identical), so the gain is small but essentially free of regressions on this difficulty. It does not generalise across difficulties: on Sudoku-easy refinement moves 99.6% to 99.7%, and on Sudoku-medium it lowers the solve rate from 96.7% to 95.9% (RT_6) and 96.0% (three rounds).

Sudoku also has an exact, free verifier (rows, columns, boxes and the given clues), so extra passes can be spent until a board verifies. We compared, at equal mean generator passes, retrying from the current board with a round trip at temperature T against drawing fresh samples. Fresh sampling at τ=1 is clearly better: 89.9% at 16.8 passes per puzzle, 91.0% at 19.2 and 93.4% at 23.9, against at most 89.5–90.7% for round-trip retries at 19–22 passes; retries stay correlated with the wrong board. Resampling only the coarse plan with greedy fine levels does not beat fresh sampling either (best 92.3–93.3% at about 24 passes over seeds 1, 4, 5 and 6). NCM's Sudoku accuracy rises from 84% to 93% with verifier-guided test-time compute, but plain resampling, not round-trip refinement, is the way to spend it.

Sampler	NFE	solved %	cell %	new / broken	McNemar p
released (greedy)	11	84.1	98.13	–	–
RT_9	14	84.7	98.14	6 / 0	3.1e-02
RT_8	15	84.9	98.15	8 / 0	7.8e-03
RT_6	17	85.5	98.19	14 / 0	1.2e-04
RT_4	19	84.9	98.15	10 / 2	3.9e-02
$\text{RT}_6\times 2$	23	85.2	98.20	11 / 0	9.8e-04
$\text{RT}_6\times 4$	35	85.2	98.21	11 / 0	9.8e-04
$\text{RT}_4\times 3$	35	85.0	98.15	10 / 1	1.2e-02
RT_3+RT_6+RT_9	29	85.9	98.21	19 / 1	4.0e-05

Table 7: Sudoku-hard, released checkpoint, greedy decoding, 1,000 test puzzles. "New / broken": puzzles solved only by the refined sampler / only by the released one.

5.9 Is it a temperature schedule? Exact twins and human text

Because attention is block-diagonal, level j sees only $\hat{z}_{j-1}$, so resampling levels $\geq k$ from the draft's own coarse codes is distributed exactly like the draft. Every refinement recipe therefore has an exact 11-pass twin: the same per-level temperatures, no round trip. Table 9 compares recipes with their twins on held-out seeds 4–6, together with a temperature curve from τ=0.6 to 1.3 measured on the same seeds.

At the released temperature the twin of RT_6 is the released sampler itself, and refinement accounts for the whole −14.5%. With hot coarse and cool fine levels the picture is more nuanced: the per-level schedule alone already lies well below the global-temperature curve (−21.0% for the twin of the best recipe, −11.3% for the twin of the τ=1.2

Sampler	NFE	Gen. PPL ↓	Δ	H	dist-3 ↑	xs-4 ↓	vs. τ frontier
released (τ=1)	11	16.15±0.17	–	3.980	0.749	0.162	–
Resample_6	17	16.43±0.33	+1.7%	3.980	0.750	0.164	+2.1%
LatentRQ_6	17	14.29±0.25	-11.5%	3.966	0.741	0.170	-6.8%
RT_6	17	13.36±0.12	-17.3%	3.974	0.740	0.170	-12.4%
$\text{Resample}_5\times2$	25	16.13±0.27	-0.2%	3.982	0.745	0.171	+3.7%
$\text{LatentRQ}_5\times2$	25	12.94±0.25	-19.9%	3.963	0.725	0.188	-6.7%
$\text{RT}_5\times2$	25	11.70±0.31	-27.6%	3.970	0.718	0.191	-12.9%
τ=1.1	11	18.76±0.32	+16.1%	3.993	0.764	0.151	–
τ=1.2	11	24.01±0.95	+48.7%	3.998	0.787	0.132	–
RT_6, draft τ=1.1	17	14.15±0.21	-12.4%	3.981	0.753	0.156	-14.7%
RT_6, draft τ=1.2	17	15.28±0.30	-5.4%	3.979	0.767	0.143	-20.9%
RT_6, draft τ=1.2, $\tau_{\geq6}$=0.8	17	14.98±0.24	-7.3%	3.980	0.766	0.143	-22.0%
$\text{RT}_5\times2$, draft τ=1.1	25	12.10±0.36	-25.1%	3.974	0.733	0.177	-16.7%
$\text{RT}_5\times2$, draft τ=1.2	25	12.63±0.19	-21.8%	3.973	0.740	0.167	-17.1%
$\text{RT}_5\times2$, draft τ=1.2, $\tau_{\geq6}$=0.8	25	12.45±0.11	-22.9%	3.972	0.740	0.167	-18.1%
RT_3+RT_6+RT_9, draft τ=1.2	29	11.80±0.33	-27.0%	3.964	0.727	0.184	-17.3%
LM1B large (FLM protocol)							
released (τ=1)	11	80.11±0.34	–	4.339	0.895	0.043	–
Resample_6	17	80.60±2.00	+0.6%	4.339	0.896	0.043	+2.4%
LatentRQ_6	17	65.78±0.54	-17.9%	4.313	0.874	0.055	-7.7%
RT_6	17	62.46±1.63	-22.0%	4.338	0.878	0.052	-20.9%
$\text{RT}_6\times2$	23	56.63±0.88	-29.3%	4.346	0.869	0.059	–
$\text{RT}_6\times2$, draft τ=1.1	23	60.04±1.24	-25.0%	4.357	0.881	0.049	–
$\text{RT}_6\times2$, draft τ=1.2	23	62.47±0.37	-22.0%	4.367	0.894	0.040	–

Table 8: Ablations and hot drafts (TinyStories, GPT-2 Large, 3 seeds × 256; LM1B large, FLM protocol, same seeds). Resample: regenerate without re-encoding. LatentRQ: re-quantize the latent instead of re-encoding the text. Draft τ: temperature of the whole draft; $\tau_{\geq6}$: temperature of levels 6–11 (of the rounds only when a draft temperature is given, otherwise of the draft and the rounds). Last column: Gen. PPL relative to the temperature curve (τ=0.7..1.2 on TinyStories, 0.8..1 on LM1B) at the same distinct-3 (TinyStories) or entropy (LM1B); "–" where a variant lies outside the curve.

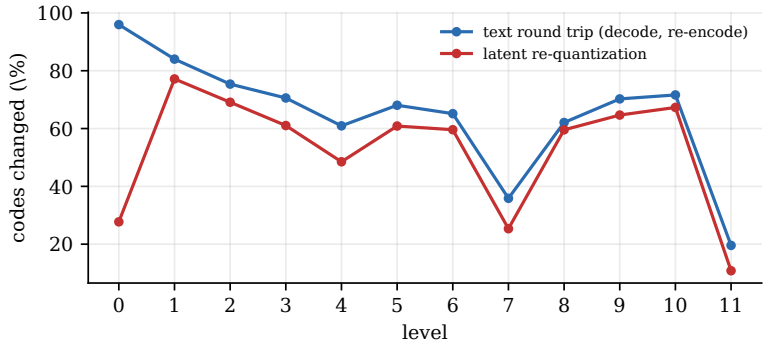


Figure 5: Share of codes changed per level when the draft's stack is replaced by the tokenizer's stack for the same text (blue) or by the re-quantized latent (red), TinyStories, 768 samples.

recipe), and refinement adds 4–10 points on top (−25.2%, −21.4%). We therefore report the hot-draft samplers as "schedule plus round trip" and attribute to refinement only its increment. On LM1B large the twin of the hot-draft sampler is far worse at a similar entropy (90.27 vs. 63.37, entropy 4.361 vs. 4.368).

Applied to real human text instead of a draft, refinement does not lower Gen. PPL below that of the human text: one round raises it from 8.51 to 9.39 with distinct-3 and cross-sample overlap essentially unchanged (0.772 vs. 0.775, 0.141 vs. 0.139), and further rounds raise it more. A sharpening operator would push human text to lower perplexity and lower diversity; round-trip refinement instead moves the generator's drafts towards the text it was trained on and leaves real text close to where it is, apart from the generator's own realisation errors.

Sampler	NFE	Gen. PPL ↓	dist-3 ↑	xs-4 ↓	vs. τ frontier
released (τ=1)	11	16.32	0.748	0.165	–
RT_6	17	13.23	0.737	0.173	-14.5%
RT_5×2, draft τ=1.3, $\tau_{\geq 6}$=0.8 (best recipe)	25	12.96	0.754	0.153	-25.2%
$\tau_{\geq 1}$=1.3, $\tau_{\geq 5}$=1, $\tau_{\geq 6}$=0.8 (its twin)	11	17.99	0.778	0.133	-21.0%
RT_5×2, draft τ=1.2, $\tau_{\geq 6}$=0.8 (recipe)	25	12.47	0.742	0.165	-21.4%
$\tau_{\geq 1}$=1.2, $\tau_{\geq 5}$=1, $\tau_{\geq 6}$=0.8 (its twin)	11	17.10	0.764	0.147	-11.3%
RT_6, $\tau_{\geq 1}$=1.2, $\tau_{\geq 6}$=0.8 (recipe)	17	14.04	0.752	0.155	-17.1%
$\tau_{\geq 1}$=1.2, $\tau_{\geq 6}$=0.8 (its twin)	11	17.73	0.766	0.146	-10.2%
τ=0.6	11	12.10	0.681	0.232	–
τ=0.7	11	12.65	0.699	0.214	–
τ=0.8	11	13.44	0.717	0.195	–
human text	0	8.51	0.775	0.139	–
human + RT_6	6	9.39	0.772	0.141	–
human + RT_5×2	14	10.35	0.760	0.151	–
human + RT_3+RT_6+RT_9	18	12.12	0.750	0.163	–

Table 9: Refinement recipes and their exact 11-pass twins (same per-level temperatures, no round trip), TinyStories held-out seeds 4–6. "$\tau_{\geq k}$" sets the temperature of levels $\geq k$; parts apply in order. Human rows apply the rounds to 768 real validation openings instead of a draft.

5.10 Does it transfer to images?

Next-scale image models face the same structural situation: VAR [30] samples a stack of multiscale residual codes and decodes it with a VQ-VAE. We applied the round trip to the released VAR-d16 ImageNet-256 model with its FID sampling settings, copying its sampler verbatim and only forcing the coarse scales: decode the draft to an 8-bit image, re-encode it with VAR's own VQ-VAE, keep scales $< k$ and regenerate the rest (Table 10; 10 images per class, FID-10k against the ADM reference images with Inception features from `pytorch-fid`). Forcing every scale to the draft's own codes reproduces all 2,000 test drafts exactly. The phenomenon carries over: the image round trip changes 29–92% of the codes at the first nine scales. The benefit does not: the best restart moves FID from 9.13 to 8.92, while resampling without re-encoding, which draws from the released distribution, already moves it to 9.22, so differences of 0.1–0.2 are within noise, and restarting from coarser scales is worse. Unlike text, the pixel round trip is lossy, and VAR conditions every scale on all earlier scales rather than on the latest reconstruction only; either may explain why canonical coarse codes do not help there. We did not run the 50k-sample confirmation.

Sampler (VAR-d16, ImageNet 256)	FID-10k ↓	codes changed by the round trip
VAR sampler (cfg 1.5, top-k 900, top-p 0.96)	9.13	–
Resample_7 (no re-encode)	9.22	–
LatentRQ_7	9.26	26–92%
RT_8	8.92	29–92%
RT_7	9.13	29–92%
RT_6	9.30	29–92%
RT_5	9.35	29–92%
$RT_7 \times 2$	9.25	29–92%

Table 10: Round-trip refinement on an image next-scale model. The round trip goes through an 8-bit image. Code changes are per scale, over the first nine of ten scales.

6 What did not work

Before finding round-trip refinement we tried six training-side ideas on the TinyStories checkpoint, two of which the NCM paper names as future work. All fine-tunes continue the released generator with AdamW (0.9, 0.95), weight decay 0.1 and gradient clipping 1, as in the paper, but with batch 64–128 instead of 2,048 for budget reasons. Table 11 summarises them; numbers are GPT-2 Large Gen. PPL at full depth unless stated.

The distillation result deserves a comment, because without diversity metrics it looks like a near-win: against

Idea	Outcome
Early exit (adaptive depth)	No plateau to exit into: the last pass alone takes GPT-2 small Gen. PPL from 37 to 20.5; the best per-sample stopping rule saves 0.7 passes for +3%.
Level skipping at inference	One skipped middle level +6–11%, three +34%, five 4.9–7×, although the tokenizer still reconstructs 98–99% of tokens from every other level.
Level-dropout fine-tuning	With the paper's conditioning corruption, no gain at 8–10 passes; wins only at 6 passes (54.6 vs. 79.7), still 3.4× worse than full depth. Without corruption, any fine-tune collapses to 36–40 (exposure bias).
Joint tokenizer + generator	Tie: 16.83 (with a coupling loss) vs. 16.85 for the generator-only control; reconstruction stays 99.98%.
Own-error corruption	Replacing uniform corruption by the model's own sampled guesses hurts, dose-dependently: 21.8 (half) and 40.2 (all) vs. 16.85.
Distilling refinement	Training on 50% refined stories matches the released 16.15 at 11 passes, but at the diversity of τ=0.8 (Table 12).

Table 11: Training-side ideas that did not beat the released sampler.

Generator (11 passes, τ=1)	batch	Gen. PPL ↓	dist-3 ↑	xs-4 ↓
released (no fine-tune)	–	16.15	0.749	0.162
control fine-tune	128	16.78	0.755	0.158
control, soup	128	16.43	0.750	0.162
distilled	128	16.15	0.719	0.188
distilled, soup	128	16.00	0.734	0.176
released, τ=0.8	–	13.34	0.719	0.191

Table 12: Distilling round-trip refinement into the generator: 1,500 fine-tuning steps on half real stories and half RT_5×2-refined stories, against a control on real stories only; "soup" averages the fine-tune with the released weights [33]. The distilled model ties the released Gen. PPL only because it is less diverse: the released sampler at the same distinct-3 (τ=0.8) scores 13.34.

its own control the distilled model is 3.8% better on every seed, and fine-tuning drift (the control is 3.9% worse than the released model) seems to be the only obstacle. Corpus diversity tells a different story. Sequence-level distillation [11] transferred the refined sampler's lower diversity (distinct-3 0.719 vs. 0.749) but not its fluency at that diversity: at matched distinct-3 the distilled model is about 21% worse than simply lowering the released model's temperature. A larger batch would reduce the drift but not this gap. The common thread of Table 11 is that, at this scale, inference-time changes to the released model were far cheaper and more effective than any fine-tune we could afford.

7 Related work

Coarse-to-fine and residual-code generation. NCM [13] brings next-scale prediction [12, 30] and residual quantization [3, 14] to text. Infinity's bitwise self-correction [10] re-quantizes *training* targets after flipping input bits so that later scales learn to fix earlier errors; our method instead re-quantizes *at inference*, needs no retraining, and goes through the decoded tokens.

Re-quantize and regenerate. The closest prior method is scale-travel in DiverseVAR [21]: re-quantize the accumulated image latent, rewind to a coarser scale and regenerate, used there to restore quality after diversity-boosting perturbations; no base-sampler result is reported. Round-trip refinement differs in going through the discrete output (decode, then re-encode) rather than re-quantizing the latent, in targeting text, and in being evaluated as a base sampler with frontier controls; LatentRQ_k (Section 5.4) is the direct comparison; on NCM it recovers about half of the matched-diversity gain of the text round trip. SDEdit [19] similarly re-noises an output and denoises it again with a diffusion model, and Restart sampling [34] alternates forward noising and backward sampling to correct accumulated errors, the closest mechanism in continuous diffusion. TTS-VAR [6] spends test-time compute on scale-wise image generation through selection.

Iterative refinement and self-correction. Mask-Predict [9], MaskGIT [4], Token-Critic [16] and ReMDM re-masking [31] revise parallel predictions by re-masking and re-predicting tokens; Self-Refine [18] has large LMs

critique and rewrite their own outputs. In NCM, codes within a level are conditionally independent given the coarser levels, so re-masking within a level does not apply; the round trip changes the *conditioning* instead.

Exposure bias. Training on model outputs [1, 29] and student-forcing for scale-wise image models [36] address the train–test gap in training; our own-error corruption experiment is in this family and did not help in fine-tuning. Joint or post-training of tokenizers [15, 25, 32] helps in vision; for NCM text at our scale it tied.

Evaluation. Gen. PPL can be lowered by reducing diversity or by low-precision sampling [8, 24, 35]; we therefore report temperature frontiers and corpus diversity alongside it.

8 Limitations

Our evaluation relies on Gen. PPL under GPT-2 Large, simple diversity statistics, MAUVE and a single 7B LLM judge with a strong position bias that we average out; we did not run a human evaluation, and at equal Gen. PPL the judge does not distinguish refinement from a lower temperature. All models are around 100M parameters with 128-token sequences; the OpenWebText checkpoint, the only released model of long general text, was not yet available on the hub when we ran these experiments. Refinement costs 1.3–2.6× the generator passes and 1.5–3.3× the generator FLOPs. It fails when the tokenizer round trip is lossy (WMT14), and it does not transfer to the image model we tried (VAR-d16, FID-10k only). Our reproduction of the released TinyStories sampler is 5% above the paper's number, and all relative claims are against our reproduction. Restart levels and hot-draft temperatures were chosen on the same three seeds we report, from small grids. Fine-tuning baselines were limited to batch 64–128, so the training-side negative results may not hold at the paper's batch size, with the exception of the diversity argument against distillation.

9 Conclusion

NCM's coarse-to-fine sampler produces code stacks that its own tokenizer would never produce for the same text. Passing the draft through the tokenizer and regenerating the fine levels repairs this, with no training and a few lines of code, and lowers generative perplexity by 17–31% on TinyStories and 22–29% on LM1B while keeping diversity, 10–14% beyond what temperature can buy and 15–22% when paired with a hotter draft. The ablations locate the gain in the re-encoding itself: resampling without it does nothing, and re-quantizing the latent instead of the text gives about half. The results hold on held-out seeds and do not shrink with model size; exact temperature twins and real human text show that the gain is neither a temperature schedule nor a sharpener; an LLM judge prefers refined stories; MAUVE moves closer to human text; and on Sudoku the gain is limited to the hardest puzzles, while verifier-guided search is better served by plain resampling. The method needs a lossless tokenizer round trip, and it trades extra passes for quality, so it complements rather than replaces NCM's main appeal of speed. Training-time alternatives that we could afford, including distilling the refined sampler, did not help.

Acknowledgements. We thank the NCM authors for releasing checkpoints, data and evaluation code, without which none of this would have been possible. This work was carried out independently of any King's College London course. Code and experiments were developed with the assistance of an AI coding assistant (Claude); the author is responsible for all content.

References

[1] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. *arXiv preprint arXiv:1506.03099*, 2015.

[2] Ondřej Bojar, Christian Buck, Christian Federmann, Barry Haddow, Philipp Koehn, Johannes Leveling, Christof Monz, Pavel Pecina, Matt Post, Herve Saint-Amand, Radu Soricut, Lucia Specia, and Aleš Tamchyna. Findings of the 2014 workshop on statistical machine translation. In *Proceedings of the Ninth Workshop on Statistical Machine Translation*, 2014.

[3] Zalán Borsos, Matt Sharifi, Damien Vincent, Eugene Kharitonov, Neil Zeghidour, and Marco Tagliasacchi. SoundStorm: Efficient parallel audio generation. *arXiv preprint arXiv:2305.09636*, 2023.

[4] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T. Freeman. MaskGIT: Masked generative image transformer. *arXiv preprint arXiv:2202.04200*, 2022.

[5] Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge, Thorsten Brants, Phillipp Koehn, and Tony Robinson. One billion word benchmark for measuring progress in statistical language modeling. *arXiv preprint arXiv:1312.3005*, 2013.

[6] Zhekai Chen, Ruihang Chu, Yukang Chen, Shiwei Zhang, Yujie Wei, Yingya Zhang, et al. TTS-VAR: A test-time scaling framework for visual auto-regressive generation. *arXiv preprint arXiv:2507.18537*, 2025.

[7] Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023.

[8] Antonio Franca and Alexander Tong. Hacking generative perplexity: Why unconditional text evaluation needs distributional metrics. *arXiv preprint arXiv:2606.08417*, 2026.

[9] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. Mask-predict: Parallel decoding of conditional masked language models. *arXiv preprint arXiv:1904.09324*, 2019.

[10] Jian Han, Jinlai Liu, Yi Jiang, Bin Yan, Yuqi Zhang, Zehuan Yuan, Bingyue Peng, and Xiaobing Liu. Infinity: Scaling bitwise autoregressive modeling for high-resolution image synthesis. *arXiv preprint arXiv:2412.04431*, 2024.

[11] Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. *arXiv preprint arXiv:1606.07947*, 2016.

[12] Hermann Kumbong, Xian Liu, Tsung-Yi Lin, Ming-Yu Liu, Xihui Liu, Ziwei Liu, Daniel Y. Fu, Christopher Ré, and David W. Romero. HMAR: Efficient hierarchical masked auto-regressive image generation. *arXiv preprint arXiv:2506.04421*, 2025.

[13] Hermann Kumbong, Ishan S. Khare, Iddah Mlauzi, Andrew Woen, Charles Ding, E. Kelly Buchanan, Simran Arora, Azalia Mirhoseini, and Christopher Ré. Next-concept modeling: Fast, parallel coarse-to-fine text generation. *arXiv preprint arXiv:2610.14162*, 2026.

[14] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. Autoregressive image generation using residual quantization. *arXiv preprint arXiv:2203.01941*, 2022.

[15] Xingjian Leng, Jaskirat Singh, Yunzhong Hou, Zhenchang Xing, Saining Xie, and Liang Zheng. REPA-E: Unlocking VAE for end-to-end tuning with latent diffusion transformers. *arXiv preprint arXiv:2504.10483*, 2025.

[16] José Lezama, Huiwen Chang, Lu Jiang, and Irfan Essa. Improved masked image generation with token-critic. *arXiv preprint arXiv:2209.04439*, 2022.

[17] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, 2004.

[18] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, et al. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.

[19] Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu, and Stefano Ermon. SDEdit: Guided image synthesis and editing with stochastic differential equations. *arXiv preprint arXiv:2108.01073*, 2021.

[20] Shashi Narayan, Shay B. Cohen, and Mirella Lapata. Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. *arXiv preprint arXiv:1808.08745*, 2018.

[21] Mingue Park, Prin Phunyaphibarn, Phillip Y. Lee, and Minhyuk Sung. DiverseVAR: Balancing diversity and quality of next-scale visual autoregressive models. *arXiv preprint arXiv:2511.21415*, 2025.

[22] Krishna Pillutla, Swabha Swayamdipta, Rowan Zellers, John Thickstun, Sean Welleck, Yejin Choi, and Zaid Harchaoui. MAUVE: Measuring the gap between neural text and human text using divergence frontiers. *arXiv preprint arXiv:2102.01454*, 2021.

[23] Matt Post. A call for clarity in reporting BLEU scores. *arXiv preprint arXiv:1804.08771*, 2018.

[24] Patrick Pynadath, Jiaxin Shi, and Ruqi Zhang. Generative frontiers: Why evaluation matters for diffusion language models. *arXiv preprint arXiv:2604.02718*, 2026.

[25] Kai Qiu, Xiang Li, Hao Chen, Jason Kuen, Xiaohao Xu, Jiuxiang Gu, Yinyi Luo, Bhiksha Raj, et al. Image tokenizer needs post-training. *arXiv preprint arXiv:2509.12474*, 2025.

[26] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[27] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[28] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *arXiv preprint arXiv:2406.07524*, 2024.

[29] Nikolay Savinov, Junyoung Chung, Mikolaj Binkowski, Erich Elsen, and Aaron van den Oord. Step-unrolled denoising autoencoders for text generation. *arXiv preprint arXiv:2112.06749*, 2021.

[30] Keyu Tian, Yi Jiang, Zehuan Yuan, Bingyue Peng, and Liwei Wang. Visual autoregressive modeling: Scalable image generation via next-scale prediction. *arXiv preprint arXiv:2404.02905*, 2024.

[31] Guanghan Wang, Yair Schiff, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Remasking discrete diffusion models with inference-time scaling. *arXiv preprint arXiv:2503.00307*, 2025.

[32] Hanyu Wang, Saksham Suri, Yixuan Ren, Hao Chen, and Abhinav Shrivastava. LARP: Tokenizing videos with a learned autoregressive generative prior. *arXiv preprint arXiv:2410.21264*, 2024.

[33] Mitchell Wortsman, Gabriel Ilharco, Jong Wook Kim, Mike Li, Simon Kornblith, Rebecca Roelofs, Raphael Gontijo-Lopes, Hannaneh Hajishirzi, et al. Robust fine-tuning of zero-shot models. *arXiv preprint arXiv:2109.01903*, 2021.

[34] Yilun Xu, Mingyang Deng, Xiang Cheng, Yonglong Tian, Ziming Liu, and Tommi Jaakkola. Restart sampling for improving generative processes. *arXiv preprint arXiv:2306.14878*, 2023.

[35] Kaiwen Zheng, Yongxin Chen, Hanzi Mao, Ming-Yu Liu, Jun Zhu, and Qinsheng Zhang. Masked diffusion models are secretly time-agnostic masked models and exploit inaccurate categorical sampling. *arXiv preprint arXiv:2409.02908*, 2024.

[36] Gengze Zhou, Chongjian Ge, Hao Tan, Feng Liu, and Yicong Hong. Rethinking training dynamics in scale-wise autoregressive generation. *arXiv preprint arXiv:2512.06421*, 2025.

A Implementation

Refinement reuses the released `NextConceptTransformer.generate`. The only change is a wrapper around its per-level sampling function:

```
_orig = net._sample_level
def _sample_with_force(logits, temperature, top_k, top_p):
    level = state["calls"] + 1 - state["offset"]   # level-0 queries counted by offset
    state["calls"] += 1
    if state["codes"] is not None and 1 <= level < state["upto"]:
        return state["codes"][level]               # re-encoded codes for levels < k
    return _orig(logits, temperature, top_k, top_p)
net._sample_level = _sample_with_force

draft = generate(l0)                                # authors' sampler, unchanged
codes = split_levels(tokenizer.vqvae.encode(draft)) # re-encode the decoded tokens
refined = generate(codes[0], forced=codes, upto=k)  # regenerate levels k..K-1
```

All scripts, run logs and per-seed results are in the accompanying repository.

B Samples

TinyStories, seed 1, sample 4 (same draft for all rows; differences from the draft in bold).

Draft (released sampler). One day, a little cat named Amy went to a big room to play. In the room, there were many toys **fun There** were games, and Amy felt happy. She looked and saw a cat. The cat was happy too. The cat tried to catch Amy, but she did not know how. The cat tried to jump high **to to** reach the cat. Amy felt sad and missed the sad cat. …

RT_6**.** One day, a little cat named Amy went to a big room to play. In the room, there were many toys**. There** were games, and Amy felt happy. She looked and saw a cat. The cat was happy too. The cat tried to catch Amy, but she did not know how. The cat tried to jump **up high to** reach the cat. Amy felt sad and missed the sad cat. …

RT_3+RT_6+RT_9**.** One day, a little girl named Amy went to the gym to play. In the gym, there were many kids laughing and playing games. Amy was very happy sound. She looked around and saw a furry cat. The cat was very sad. Amy wanted to help the cat, but she did not know how. She tried to jump high, but she fell down too. …

A restart at level 6 keeps the story and repairs local errors; a restart at level 3 re-plans the content ("cat named Amy" becomes "girl named Amy") and, as Table 1 shows, trades a little diversity for the largest fluency gain.